

TECNICO.md

1. Visão Geral da Arquitetura

Este projeto implementa um RAG (Retrieval-Augmented Generation) para domínio de seguro residencial com os seguintes blocos principais:

1. Interface web (Streamlit) para chat e upload de PDFs.
 2. Pipeline de ingestão que transforma PDFs em chunks textuais + embeddings.
 3. Banco PostgreSQL com pgvector para armazenamento de chunks e vetores.
 4. Camada de recuperação híbrida (**dense** + **sparse** + **hybrid/RRF**).
 5. Camada de geração (LLM) com resposta fundamentada nos chunks recuperados.
 6. Guardrail de suficiência de evidência para mitigar respostas sem base.
-

2. Componentes e Responsabilidades

2.1 Frontend e Orquestração de UI

Arquivo principal: **app.py**

Responsabilidades:

- Exibir chat e streaming de resposta.
- Exibir fontes recuperadas por resposta.
- Upload de PDF com validações:
 - somente extensão **.pdf**;
 - tamanho máximo configurado no código (50 MB);
 - bloqueio de nome duplicado.
- Ao subir arquivo:
 - salvar em **documentos/** (pasta operacional);
 - disparar ingestão automática;
 - informar sucesso/erro.
- Botão de limpeza total:
 - remove PDFs de **documentos/**;
 - executa **TRUNCATE TABLE dados RESTART IDENTITY**.

2.2 Ingestão

Arquivos:

- **rag/ingest/extract_text.py**
- **rag/ingest/pgvector_store.py**
- **rag/settings.py**

Responsabilidades:

- Ler PDF com **pypdf**.

- Normalizar e segmentar texto em chunks com overlap.
- Extrair metadados básicos por arquivo/chunk.
- Gerar embeddings (local/Ollama ou OpenAI).
- Persistir no PostgreSQL (**dados**) com deduplicação por **chunk_id**.

2.3 Recuperação

Arquivos:

- **rag/retrieval/retriever.py**
- **rag/retrieval/retrieval_node.py**

Responsabilidades:

- Carregar todos os chunks do banco.
- Índice sparse com BM25.
- Busca vetorial dense via produto interno.
- Fusão híbrida via Reciprocal Rank Fusion (RRF).
- Expansão opcional de query para termos do domínio.

2.4 Geração e Prompting

Arquivos:

- **rag/augmented/augmented_node.py**
- **rag/graph/prompt.py**
- **rag/graph/model_provider.py**

Responsabilidades:

- Construir prompt final com pergunta + contexto recuperado.
- Gerar resposta por streaming de tokens.
- Manter fidelidade ao contexto textual recuperado.

2.5 Guardrails de Evidência

Arquivo:

- **rag/augmented/evidence_guard.py**

Responsabilidades:

- Avaliar se os chunks recuperados parecem suficientes para responder.
- Usar heurística de cobertura de palavras-chave.
- Opcionalmente usar LLM como juiz de suficiência.
- Sinalizar recusa quando contexto for insuficiente.

2.6 Grafo de Execução

Arquivo:

- **rag/graph/rag_graph.py**

Responsabilidades:

- Orquestrar pipeline com LangGraph:
 - nó `retrieve`
 - nó `generate`
 - Emitir eventos para UI:
 - `details`
 - `token`
 - `sources`
-

3. Pipeline Fim a Fim (Fluxo Principal)

3.1 Fluxo de Upload + Ingestão

1. Usuário envia PDF pela sidebar.
2. `app.py` valida extensão/tamanho/nome duplicado.
3. Arquivo (normalmente selecionado de `documentos_para_upload/`) é salvo em `documentos/<nome>.pdf`.
4. `app.py` chama ingestão automática:
 - `process_pdf_file()` em `extract_text.py`
 - chunking + metadados
 - `embed_text()` para cada chunk
 - `PgVectorStore.add_texts()`
5. `pgvector_store.py` sanitiza `\x00` em texto/metadados.
6. Registros são inseridos em `dados` com `ON CONFLICT (chunk_id) DO NOTHING`.

Saída:

- novos chunks disponíveis para recuperação imediatamente.

3.2 Fluxo de Pergunta/Resposta

1. Usuário envia pergunta no chat.
2. `run_streaming_rag()` cria estado inicial (`question, retrieval_mode, top_k`).
3. Nó `retrieve` executa:
 - query original;
 - query expandida (quando habilitada);
 - busca `dense, sparse, hybrid`.
4. UI recebe evento `details` com metadados da busca.
5. Nó `generate` constrói resposta baseada no contexto recuperado.
6. UI recebe tokens em streaming (`token`).
7. Ao final, UI recebe `sources` para exibir rastreabilidade.

Saída:

- resposta textual + lista de fontes/chunks usados.
-

4. Modelo de Dados

4.1 Tabela principal: **dados**

Criada/garantida em `PgVectorStore.create_table()` e `HybridRetriever._ensure_schema()`.

Campos:

- `id` SERIAL PRIMARY KEY
- `chunk_id` TEXT UNIQUE
- `doc_id` TEXT
- `titulo` TEXT
- `fonte` TEXT
- `data` TEXT
- `tipo` TEXT
- `text` TEXT
- `metadata` JSONB
- `embedding` VECTOR(PGVECTOR_DIM)

Regras importantes

- `chunk_id` é a chave lógica de deduplicação.
 - Inserção usa `ON CONFLICT DO NOTHING`.
 - `metadata` replica informações úteis para rastreabilidade e ranking.
-

5. Estratégia de Chunking

Implementada em `extract_text.py`.

Parâmetros (via `.env`):

- `CHUNK_SIZE`
- `CHUNK_OVERLAP`

Processo:

1. Texto extraído e normalizado de espaços.
 2. Seções naturais detectadas por marcadores do domínio (ex.: coberturas, exclusões, franquia, sinistro).
 3. Divisão por tamanho alvo + overlap quando necessário.
 4. Metadados por chunk (`chunk_type`, `chunk_index`, etc.).
-

6. Estratégia de Recuperação

6.1 Dense

- Embedding da query via `embed_text()`.
- Similaridade por produto interno com embeddings salvos.

6.2 Sparse

- Tokenização (normalização + stopwords + stemmer RSLP).

- BM25 sobre todos os chunks carregados.

6.3 Hybrid

- Fusão de ranking entre dense e sparse com RRF.

6.4 Query Expansion

- `retrieval_node._expand_query()` usa LLM para reformular pergunta no vocabulário de seguro residencial.
 - Quando a expansão difere da original, os resultados de ambas são fundidos.
-

7. Estratégia de Geração

- Contexto recuperado é passado ao prompt de geração.
 - Resposta é transmitida por streaming para UX mais responsiva.
 - Fontes são mostradas no final, com metadados de origem.
-

8. Guardrails / Confiabilidade

`evidence_guard.py` combina:

1. heurística lexical (keywords da pergunta vs texto recuperado);
2. sinais de quantidade/qualidade de documentos;
3. validação adicional por LLM (quando aplicável).

Objetivo:

- reduzir alucinação;
 - permitir recusa quando evidência for fraca.
-

9. Modos de Modelo (Local vs OpenAI)

Arquivo central: `rag/graph/model_provider.py`

9.1 Local (`MODELO_LOCAL=true`)

- LLM: Ollama (`LLM_MODEL`)
- Embedding: Ollama (`EMBEDDING_MODEL`)
- URL configurável por `OLLAMA_URL`

9.2 OpenAI (`MODELO_LOCAL=false`)

- Chat: `OPENAI_MODEL`
 - Embedding: `OPENAI_EMBEDDING_MODEL`
 - Chave: `OPENAI_API_KEY`
 - Embeddings respeitam `PGVECTOR_DIM`.
-

10. Execução em Containers

Arquivos:

- `docker-compose.yaml`
- `Dockerfile`

Serviços:

1. `postgres` (`rag_insurminds_postgres`)
 - imagem `pgvector/pgvector:pg16`
 - porta host `127.0.0.1:54321 -> 5432`
 - volume persistente `postgres_data`
2. `streamlit` (`rag_insurminds_streamlit`)
 - build local
 - porta `8501`
 - monta `./documentos:/app/documentos`
 - pasta recomendada de origem para seleção na UI: `./documentos_para_upload`

Observações:

- o mount de `documentos` garante que uploads apareçam no host;
 - o app no container conversa com banco via host interno `postgres:5432`.
-

11. Operações de Manutenção

11.1 Limpeza via UI

Botão "Remover todos os arquivos":

- remove PDFs de `documentos/`;
- limpa tabela `dados`.

11.2 Limpeza via CLI

- `python -m rag.ingest.truncate_data`
- `python -m rag.ingest.reset_project_state`

11.3 Reingestão completa

1. limpar base;
 2. garantir PDFs em `documentos/`;
 3. rodar `python -m rag.ingest.extract_text`.
-

12. Pontos Críticos para o Desenho Técnico

Para seu diagrama, represente explicitamente:

1. **Canal de entrada**

- Upload PDF (UI) -> **documentos/**

2. Pipeline de ingestão

- PDF -> extração -> chunking -> embedding -> **dados**

3. Storage

- PostgreSQL + pgvector (**dados**)

4. Pipeline de consulta

- pergunta -> retrieve (dense/sparse/hybrid) -> geração -> resposta + fontes

5. Guardrail

- nó de suficiência de evidência antes/durante geração

6. Operações

- botão de limpeza total (filesystem + banco)

7. Infra

- container **streamlit** + container **postgres** + mount **documentos**

13. Sequência Técnica Recomendada (Mermaid-ready)

Sugestão de lifelines para diagrama de sequência:

- **Usuario**
- **Streamlit UI**
- **Ingest Service (app.py + extract_text.py)**
- **Embedding Provider (Ollama/OpenAI)**
- **PostgreSQL/pgvector**
- **Retriever**
- **Generator/LLM**

Fluxo de upload: **Usuario** -> **UI** -> **Ingest** -> **Embedding** -> **PostgreSQL**

Fluxo de pergunta: **Usuario** -> **UI** -> **Retriever** -> **PostgreSQL** -> **Generator** -> **UI** -> **Usuario**

14. Riscos Técnicos e Mitigações

1. PDFs com texto inválido (**\x00**)
 - mitigação: sanitização no **pgvector_store.py**.
2. Divergência de dimensão de embedding
 - mitigação: validação contra **PGVECTOR_DIM** antes de inserir.
3. Latência alta em embedding/LLM
 - mitigação: streaming de resposta e monitoramento de timeout.
4. Inconsistência entre arquivos e base
 - mitigação: botão de limpeza total e reingestão determinística.

15. Resumo Executivo da Pipeline

- O sistema recebe PDFs e transforma em conhecimento indexado (chunks + vetores).
- Em cada pergunta, combina busca lexical e vetorial para recuperar contexto relevante.
- A resposta é gerada com base no contexto recuperado e expõe fontes para rastreabilidade.
- A arquitetura é containerizada, com persistência em PostgreSQL/pgvector e operação simples via UI e CLI.